

DATE: ___/___/___

Name:- Hammad Sajjad Awan.

SAPID:- 76142

Subject:- Data Structure

Department:- BSDS3-1



FinTech Trading Management System

*Project Description:

The FinTech Trading Management System is a menu-driven Java Program that simulates basic tasks involved in managing financial assets and trading activity. It lets users view sample market assets, search for assets by ID, sort assets by price, manage a watchlist, calculate portfolio value, record transactions, and manage trading orders.

The project uses predefined sample assets and prices. Users can buy or sell asset quantities in the simulated portfolio while transactions are stored in a stack and orders are handled through a queue. The program demonstrates java programming concepts, including arrays, search and sorting algorithms, recursion, linked lists, stacks, and queue. It is an educational simulation and does not connect to live financial markets or perform real trades.

DATE: ___/___/___

Data Structure and Algorithms Used

* Arrays:-

The program stores the market assets in an array and uses a matching quantity array to track how many of each asset are in the portfolio. Sorting operations use a cloned array, so the original market list stays in its original order.

* Linear Search:-

This searches for an asset ID by checking the middle each asset ID from the beginning of the array. It works even if the assets are not sorted, but may need to check every asset.

* Binary Search:-

This search for an asset ID by checking the middle of the array and then narrowing the search to one half. It is faster than Linear Search for a large sorted list. The asset IDs in this project are initially in ascending order, which allows binary search to work.

* Bubble Sort:-

This sorts assets by price by repeatedly comparing neighbouring assets and swapping them when they are

DATE: ___/___/___

the wrong order. After several phases, the assets are arranged from lower price to higher price.

* Selection Sort:-

This repeatedly finds the lowest-priced asset in the unsorted part of the array and moves it to the next correct position.

* Insertion Sort:-

This builds a sorted section one asset at a time. Each asset is placed in the correct position among the assets that have already been sorted.

* Merge Sort:-

This divides the array into smaller parts until each part has one asset, then merge the parts back together in price order.

* Quick Sort:-

This choose a pivot asset places lower-priced assets before it and higher-priced assets after it, then repeats the process on each side of the pivot.

* Linked List:-

The watchlist uses a Linked List. Each node stores an asset and a reference to the next node. This structure supports adding, searching, displaying, and deleting watched assets.

DATE: ___/___/___

* Recursion:-

The portfolio methods calls themselves with the next array index. This is used to display each portfolio holding and calculate the total portfolio value. The recursion stops when it reaches the end of the array.

* Stack:-

Transactions are stored in a stack, which follows last in, first out (LIFO) order. This means the most recently added transaction is the first one removed.

* Queue:-

Trading orders are stored in a queue, which follows first in, first out () order. This means the earliest order added is the first one handled.



Dry Run:-

① Binary Search for Asset ID

105:-

It starts in ascending order:
101, 102, 103, 104, 105, 106, 107, 108, 109, 110.

Target 105

DATE: ___/___/___

Step 1:-	1
Low Index	0
High Index	9
Middle Index	4
Middle asset ID	105
Action	Match found.

The middle asset has ID 105
So the search returns the Microsoft asset.

②.

Bubble sort By Price.

I use three samples from the project

Starting Order	Asset	Price.
01	Apple	225.00
02	Tesla	340.20
03	Amazon	195.75

It compares neighbouring prices and swaps them when the left price is higher

Pass	Comparison	Result
1	Apple(225.50) vs Tesla(340.20)	No Swap
2	Tesla(340.20) vs Amazon(195.75)	Swap
3	Apple(225.50) vs Amazon(195.75)	Swap

DATE: ___/___/___

Sorted order:-

Amazon	145.75
Apple	228.50
Tesla	340.20

③.

Queue Dry Run:-

The trading order queue follows FIFO: the earlier order added is removed first.

Assume the queue has space for three orders.

Step	Operation	Queue
1	Enqueue Order 1: Buy 2 Apple	Order 1
2	Enqueue Order 2: Sell 1 Tesla	Order 1 - Order 2
3	Enqueue Order 3: Buy 3 Amazon	Order 1 - Order 2 - Order 3
4	Dequeue	Order 1 is removed
5	Next order at the front	Order 2.

till it is.

Result:-

Order 1 is processed first because it was added first. After it is removed, Order 2 becomes the next order to process, and similarly this goes till 10.